Practice Session 3:

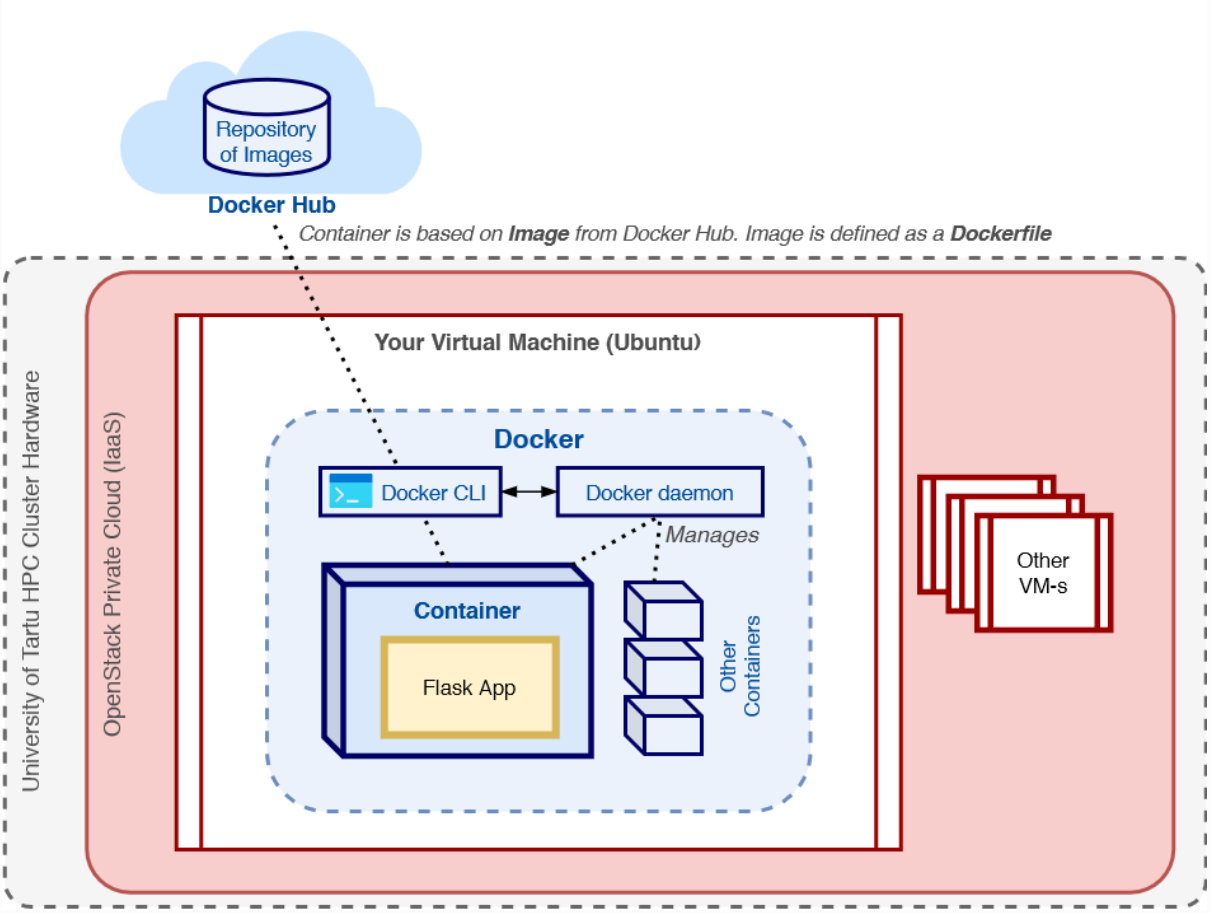
Working with Docker

© Airowire Networks All Rights Reserved

The information contained in this document is proprietary to Airowire Networks unless otherwise stated and is made available in confidence for the use of clients only. All rights reserved. The information contained in this document must not be used by or disclosed to any other person without the express permission of Airowire Networks. This document may not be copied in whole or in part without the express written consent of Airowire Networks which may be given by contract. Release of the information to any third party is prohibited without prior written consent of Airowire Networks

Practice 3 - Working with Docker

In this lab, we will take a look at how to install Docker, use Docker CLI commands, and how to containerize applications. Docker containers allow the packaging of your application (and everything that you need to run it) in a “container image”. Inside a container, you can include all necessary libraries, files and folders, environment variables, volume mount points, and your application binaries.



Key terminology:

- Docker **image**
  - Lightweight, stand-alone, executable package that includes everything needed to run a piece of software
  - Includes code, a runtime, library, environment variables, and config files
- Docker **container**
  - Runtime instance of an image - what the image becomes in memory when actually executed.
  - Completely isolated from the host environment.
- **Host** machine
  - The PC/server/VM that hosts the docker container.
  - Machine inside which docker containers run

- Docker is available in two editions: Community Edition (CE) and Enterprise Edition (EE)
- Supported Platform: macOS, Microsoft Windows 10, CentOS, Debian, Fedora, RHEL, Ubuntu and more.

References

Referred documents and websites contain supportive information for the practice.

Manuals

1. Docker fundamentals and architecture: https://docs.docker.com/engine/docker-overview/
2. Docker CLI: https://docs.docker.com/engine/reference/commandline/cli/
3. Building docker image: https://docs.docker.com/engine/reference/builder/

In case of issues check:

1. Contact to class teacher or slack channel

Exercise 3.1. Installation of docker inside OpenStack instance

In this task, you will install docker in *Ubuntu OS* and try to run the basic commands to make you comfortable with the Docker commands used in the next tasks.

- Create a virtual machine with *ubuntu22.04 OS* as carried out in **the previous Practice session** and connect to the virtual machine remotely via SSH.
  - NB! DO NOT use your previous lab image/snapshot!
- Update the apt repo `sudo apt-get update`
- Install packages to allow apt to use a repository over HTTPS:
  - `sudo apt-get install apt-transport-https ca-certificates curl gnupg-agent software-properties-common`
- Add Docker's official GPG key
  - `curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -`
- Use the following command to set up the stable repository.
  - `sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"`
- Update the apt package index, install Docker
  - `sudo apt-get update`
  - `sudo apt-get install docker-ce`

NB! To be able to run Docker commands without sudo privileges

- Add the current user to the docker group: `sudo usermod -aG docker $USER`
    - If the docker group does not exist for some reason, you can create it by using:
        - `sudo groupadd docker`
- To reload the changes for the active terminal session:
    - Exit the terminal/ssh session and rejoin (ssh) into the virtual machine.
- Check the installation by displaying docker version: `docker --version`

Exercise 3.2. Practicing docker commands

The goal of this exercise is to teach basic Docker CLI (command line interface) commands such as downloading and listing container images, attaching data volumes, running commands inside the container, checking the IP address of running containers, and forwarding ports between the container and the host machine. You can have a look into basic Docker commands here: Docker commands

We will pull an Ubuntu Docker image from Docker Hub and start an Ubuntu container in detached mode, manually install HTTP server (PS! It is not a good practice to manually install software inside containers) and use port forwarding to access container-HTTP traffic via host port 80.

- Create a login account at Docker Hub sign-up page
- Login into your docker account from the Docker host terminal: `docker login`
    - Provide input to the following:
        - Username: your docker hub id
        - Password: Docker hub password
- Pull an image from docker hub : sudo `docker pull ubuntu`
- Check the downloaded images in the local repository: `docker images`
- Run a simple ubuntu container, here detached mode (-d),-it runs interactively (so you get a pseudo-TTY with STDIN): `docker run -dit -p 80:80 --name <<yourname>> ubuntu`
    - <<yourname>> = please type your name
    - `-d` - We will run it in detached mode - container stays running in the background until stopped
    - `-p 80:80` - We will forward the inside container port 80 to the outside, host machine port 80.
- Check the running container `docker ps` (This command will list the running containers)
- Get the bash shell of the container with non-detached mode: `docker exec -it <container_name> sh`
    - Here you could start configuring your container: install packages, modify files, etc.. Alternatively, you can also run commands "outside" of the container using *docker exec*, as we will do in the next steps.

    - Exit from the container: `exit`
- Connect to container and update the apt repo: `docker exec -it <container_name> apt-get update`
- Install HTTP server: `docker exec -it <container_name> apt-get install apache2`
- Check the status of HTTP server: `docker exec -it <container_name> service apache2 status`
- If not running, start the HTTP server: `docker exec -it <container_name> service apache2 start`
- Check the webserver running container host machine: `curl localhost:80`
- Check the IP address of the container: `docker inspect <container_name> | grep -i "IPAddress"`
- Checking the logs of the container, if application is not working properly.
    - `docker logs <container_name>`
        - Usually contains the standard output of the main process running in the container
        - Some applications may still write logs into typical Linux log files (e.g., /var/log). In such cases, we can look directly into the files inside the container.
- Commit the docker container changes in to docker image `docker commit -m "added apache2 web server" -a "your_name" <container_name> "image_name"`
    - image_name : You can choose yourself
- Check the newly created image using `docker images`
- Now, stop and delete your containers using
    - `docker stop CONTAINER_NAME_OR_ID`, `docker rm CONTAINER_NAME_OR_ID`

Persisting container data: Docker isolates all files, code, and data inside the container from your local filesystem. When you delete a container, Docker deletes all the content within that container. Sometimes, you may want to persist the data that a container generates. You can create a Docker volume or mount a local directory to achieve this.

We will mount a local host directory inside the container. For example, if you store some application configuration files outside the container and mount them into the container, they can be retained even after the container is deleted and reused later. This is especially useful when configuration files contain secret information like API credentials, security tokens, etc.

- Accessing a host file system on the container with read-only and read/write modes:
    - Create directory with name test: `mkdir test && cd test`, Create a file: `touch abc.txt`

- Run a container and -v parameter to mount the host directory to the container
  - Mounting folders with read-only permissions:
    - `docker run -dit -v /home/ubuntu/test/:/data/:ro --name vol1 ubuntu sh`
  - Access the file in a container in the path /data `docker exec -it vol1 sh` than `cd /data`
  - Try listing the existing file `ls`
  - Try creating a new file (you should see access denied) from container `touch test.txt`
  - Exit from the container `exit`.
- Mounting folders with read-write permissions inside the container:
  - `docker run -dit -v /home/ubuntu/test/:/data/:rw --name vol2 ubuntu`
- Try to create some text files inside the /data folder inside the container. You should see the files created in the host machine `/home/ubuntu/test/` folder.

- NB! Run `docker ps` and take a screenshot here after running `docker ps` command
- Now, stop and delete your containers using
  - `docker stop CONTAINER_NAME_OR_ID`, `docker rm CONTAINER_NAME_OR_ID`

Exercise 3.3. Creating Docker Image using Dockerfile

The task is to create your own docker image and try to run it. We will conteinerize the simple Python web application from the previous practice session.

More information about Dockerfile (https://docs.docker.com/engine/reference/builder/). A Dockerfile is a text document that contains all the commands a user could call on the command line to assemble an image. Using docker build, users can create an automated build that executes several command-line instructions in succession.

Some of Dockerfile commands:

FROM	Set the base image
LABEL	Add a metadata to image
RUN	Execute the command and execute the commands
CMD	Allowed only once
EXPOSE	Container listen on the specific ports at runtime
ENV	Set environment variables
COPY	Copy the files or directories into container's filesystem
WORKDIR	Set working directory

The scenario is to create a docker image and deploy the flask application from Practice Session 1 (Message Board) using the docker container. In this exercise, we are going to perform three tasks:

1. Create docker **Image** for flask application (Message Board)
2. **Deploy** flask application container using the host directory as a volume (bind volumes) to store messages in the host directory (data.json in flask application).
3. **Deploy** a different version of the flask application, linking to PostgreSQL relational **database container** to store the data, instead of a JSON file.

Task 3.3.1: Create a Docker image for the flask application

- First, fetch the source code of the application with git, storing it in the folder "lab2app":
  - `git clone https://github.com/airowireNetworks/devops-web-app-2.git lab2app`
- Move into the **lab2app** directory
  - `cd lab2app`
- Modify the `home.html` file inside the templates directory.
  - Modify the HTML file like you did in the previous practice session.
  - The application should at least display your name somewhere on the page
- Now, let's containerize this application by creating a new Dockerfile - this will define the structure and content of the Docker container:
  - Create a new file `Dockerfile` (it should have no file extension) inside the `lab2app` directory
    - `nano Dockerfile`
  - Add the following set of Dockerfile commands/statements:
    - **FROM** statement specifies the base image for the container. We will use python:slim-buster, which is a thin Python image that contains the most basic Linux apps and services and the latest Python runtime. Add the following statement into the Dockerfile:
      - `FROM python:slim-buster`
    - **WORKDIR** - Sets the current working directory when your container starts. We will set the application working directory to be **/usr/src/app**:
      - `WORKDIR /usr/src/app`
    - **COPY** command will copy files from the host directory into the container (Your copying the Messageboard code into the container filesystem). To copy code from `lab2app` directory to the container working directory, add the following statements:
      - `COPY templates ./templates`
        - This moves the **templates** directory into the container working directory (indicated by ".", and which is

located in /usr/src/app/). The following statements move individual files into the working directory.

        - `COPY requirements.txt .`
        - `COPY data.json .`
        - `COPY app.py .`
        - PS! Technically, we could use `COPY lab2app .` instead to copy all files and folders in the current directory into the docker container, but it is usually better to explicitly control what files are included.
    - **RUN** command will run the specified Linux command-line command inside the container. Use the following command to install the required Python libraries requirements for our Flask app:
        - `RUN pip3 install -r requirements.txt`
    - **ENV** command will set the environment variables during the start of the container. We will specify two environment variables, one for specifying the name of the Python script to run, and the second one to specify Flask to listen to any local IP (of network interfaces) for incoming traffic:
        - `ENV FLASK_APP=app.py`
        - `ENV FLASK_RUN_HOST=0.0.0.0`
    - **EXPOSE** command can be used to specify what ports are used by the container. Selts expose port 5000:
        - `EXPOSE 5000`
    - **CMD** command will start the flask application:
        - `CMD [ "flask", "run"]`
  - NB! You can verify the correctness of your Dockerfile with this.
- Build the container image using the following command **docker build** as `docker build -t <flask_task1_lastname> .`
  - Change the lastname to your lastname
  - .(dot) indicates the path to your Dockerfile
- Now, look at the output of the build command where a set of docker commands are executed in the order you wrote in Dockerfile.
- After a successful build, you should see your built image in the list of local images when calling `docker images`
- Now, run the container with the following options:
  - port mapping: Map the host 80 port to container port 5000 (where flask is listening for traffic)
    - For port mapping, add `-p 80:5000` to the docker run command
  - Image name: `<flask_task1_lastname>`
  - Name of container: `<task1_your_lastname>`
- After this, you should see the application running at `http://VM_IP:80` through browser

- If it does not work properly, you should check the logs of the container for hints and errors indicating what could be wrong:
  - `docker logs <container_name>`
- Stop and delete the container

Task 3.3.2: Deploy Flask app container with a mounted-volume-based storage

We will deploy flask application container so that the data is stored outside the Docker container and is not lost after the container is Deleted. We will create the `data.json` file (Our Flask application stores the message board messages in this file) in a host directory and mount it into the Docker container.

- Copy the data.json file (inside the lab2app folder) into the ubuntu user home folder inside the VM in the path (`/home/ubuntu/data.json`).
- Now run the container with the following options:
  - detached mode (-d)
  - Container image `<flask_task1_lastname>`
  - Name of container `<task2_your_lastname>`
  - port mapping host(80), container(5000)
  - Volume (-v) `/home/ubuntu/data.json:/usr/src/app/data.json`
    - This will mount the file you previously created (inside the host file) into the correct location inside the container.
- By now, you should see the application running at `http://VM_IP:80` and add few messages
- Check the content of data.json `cat /home/ubuntu/data.json`, here you can see the added messages.
- Stop and remove the container
- Run the container again and should see the data of the previous container's application in http://VM_IP:80 web page
- NB! Take the screenshot of `cat /home/ubuntu/data.json` inside the VM and `docker exec -it <task2_your_lastname> cat /usr/src/app/data.json`
- Stop and delete the container

Task 3.3.3: Deploy Flask app container with a database container for storage

We will deploy another version of out Flask message board application which uses a Database instead of a local data file (data.json). We will deploy PostgreSQL relational database in a separate Docker container to store the data and link the database to our application container.

- Make a new folder for storing database data inside the host VM:

    - `sudo mkdir /mnt/data`
    - We will mount this to postgre container to store database files there
- Firstly, create a Postgresql database container using the following command and set the value of PostgreSQL configuration variable `POSTGRES_PASSWORD` (make sure to remmember the password)
    - `docker run -d --name postgres -e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=<your_password> -e PGDATA=/var/lib/postgresql/data/pgdata -v /mnt/data:/var/lib/postgresql/data -p 5432:5432 -it postgres:14.1-alpine`
- Now let us clone the modified code to store messages in the postgresql container.
    - `git clone https://github.com/airowireNetworks/devops-web-app-3.git task3lab2app`.
    - In this version of the Flask application, we use the SQLAlchemy library to enable the Flask framework to interact with Postgresql database.
- Modify the task3lab2app/templates/home.html to display your name and task number
- Create a new Dockerfile, and copy the content of the Dockerfile you created earlier in a previous task
- Remove the `COPY data.json .` line, as this application no longer uses a local data file.
- Build the image with a new name> <flask_task3_lastname>
- Now, run the container with the following options:
    - Use detached mode (-d)
    - Container image: `<flask_task3_lastname>`
    - Name of container: `<task3_your_lastname>`
    - Port mapping: host(80), container(5000)
    - Set environment variable for specifying the PostgreSQL connection URL, that contains the database credentials (-e): `DATABASE_URL=postgresql://postgres:<your_paasword>@<VM_IP>:5432/postgres`
        - The new version of the Python app expects the ENV variable `DATABASE_URL` to exist.
    - The whole command for example should look something like:
        - `docker run -d -p 80:5000 --name flask -e DATABASE_URL=postgresql://postgres:4y7sV96vA9w46VR@172.17.67.119:5432/postgres flask_poojara:latest`
        - NB! make sure to change the password and the IP address in this command to match the information of your Postgres container.
        - For IP address you must use the IP of the host VM (replacing 172.17.67.119 in the example command)
- By now, you should see the application running at http://VM_IP:80
    - Add a few messages.

- NB!! Take the screenshot of running containers (docker ps) and the web page (through browser)
- Stop and remove the postgresql container
- Create the postgresql container again
    - Check whether you can see the previous messages in the flask application.
    - The volumes we previously set up for the database container are important to persist our data in the host even when the container is deleted.

Exercise 3.4. Shipping a docker image to the Docker hub

Docker hub is a hosted repository service provided by Docker for finding and sharing container images with your team. (https://www.docker.com/products/docker-hub)

- Create a login account in the Docker hub (if you do not have one already) using the link https://hub.docker.com/signup .
- **Push the docker image to docker hub**
    - Initially login into your docker account from docker host terminal: `docker login`
        - Provide input to the following:
            - Username: your docker hub id
            - Password: Docker hub password
    - Tag your image before pushing into docker hub: `docker tag <flask_task3_lastname> your-dockerhub-id/flask1.0`
    - Finally push the image to docker hub: `docker push your-dockerhub-id/flask1.0`
- Test that you can start a new container from the DockerHub image you have just created and updated.
    - `docker run -dit -p 80:5000 --name flask -e DATABASE_URL=postgresql://postgres:4y7sV96vA9wv46VR@172.17.67.119:5432/postgres your-dockerhub-id/flask1.0:latest`
    - After replacing **your-dockerhub-id** with your own Dockerhub account